



TECHNICAL UNIVERSITY OF MUNICH

**SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY
INFORMATICS**

Master's Thesis in Informatics

**DRAFT - Thesis Proposal:
Something With Usability in PIX:E**

Rai Trouvain



TECHNICAL UNIVERSITY OF MUNICH

**SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY
INFORMATICS**

Master's Thesis in Informatics

DRAFT - Thesis Proposal:
Something With Usability in PIX:E
Thesis Proposal: Irgendetwas mit Usability
in PIX:E

Author:	Rai Trouvain
Supervisor:	Prof. Dr. Georg Groh
Advisors:	Julian Geheeb M.Sc.
Submission Date:	DRAFT - 24.07.2026

Disclaimer

I confirm that this Master's thesis is my own work and I have documented all sources and material used.

(Date)

(Rai Trouvain)

Acknowledgements

The acknowledgements

Abstract

English Abstract

Zusammenfassung

German Abstract

Contents

1 Introduction	3
2 Background	4
2.1 PaceMaker	4
2.1.1 Functionalities and Evaluation of PaceMaker	4
2.2 Lock-and-Key Puzzles	5
3 Path-Based Diagrams	6
3.1 Path Selection	6
3.1.1 Functionality	6
3.1.2 Implementation	6
3.1.3 Workflow	6
3.2 Diagram Generation	7
3.2.1 Functionality	7
3.2.2 Implementation	8
3.2.3 Workflow	8
3.2.4 Limitations	8
4 Locks and Keys	9
4.1 Definition	9
4.2 Taxonomy	9
4.2.1 Not Modeled Currently	10
4.2.2 Data Model Overview	10
4.3 Creating and Visualizing Locks and Keys	10
4.3.1 Modeling Keys	10
4.3.2 Modeling Locks	10
4.4 Pathfinding with Locks and Keys	11
4.4.1 Basic Dijkstra with Locks and Keys	11
4.4.2 Fixed Keys	13
4.4.3 Soft Gates	14
4.4.4 Consumable Keys and Soft-Lock Detection	14
4.5 Bidirectional Edges and Backtracking	16
4.5.1 Example	18
4.5.2 Full Algorithm	18
4.6 Options/Settings	19

4.7 Code Architecture	19
4.8 Testing	19
4.9 Limitations and Future Work	19
5 Evaluation	20
5.1 User Study Design	20
5.1.1 Overall Setup	20
5.1.2 Modeling Zelda Dungeons	20
5.1.3 Solvability Tasks	21
5.1.4 Pacing Analysis Tasks	21
5.1.5 Collected Data	22
5.2 User Study Results	22
5.2.1 Data Processing	22
5.2.2 Data Analysis	22
5.3 Limitations	24
5.4 Post-Study Implementations	24
5.4.1 Reachability Analysis	24
5.4.2 Increased Contrast of Nodes Against Background	24
Bibliography	25

1 Introduction

2 Background

2.1 PaceMaker

(Geheeb, Dyrda, and Geheeb 2024)

- PaceMaker: predecessor of PxCharts

“The toolkit, PaceMaker, allows the user to design a non-linear experience chart and subsequently plot relevant information like intensity or gameplay category of each node along a path on the chart.”

-> presented as prototype/PoC

“Pacing describes the rhythm that results from the recurring patterns of rhythmic parameters in time. Rhythmic parameters are divided into artifact and experience parameters.”

-> mostly numerical or categorical

2.1.1 Functionalities and Evaluation of PaceMaker

1. statecharts for modeling [(Harel 1987)]
 - nodes = beats
2. experience specification: properties that can be assigned to a beat
 - name, description, narrative/gameplay/overall intensity, gameplay category, expected playtime
 - in p:xe: PxComponents, can be defined by user (available datatypes: TODO)
3. path selection
 - start/intermediate/end beats -> Dijkstra
 - path snapshots
 - implementation in p:xe part of this work
4. pacing diagrams
 - visualize intensity or gameplay category per beat
 - in p:xe: selection based on PxComponents
 - comparison of different properties per beat
 - comparison of different paths
 - usability issues
 - nesting/concurrency

2.2 Lock-and-Key Puzzles

(Ashmore and Nitsche 2007) “The puzzle is finding out what is an obstacle, what and where is a key to overcome it, and finally using the key to master the challenge.”

(Ashmore and Nitsche 2007) “Obstacles may not be passed until the player obtains some token (such as an item or skill)”

-> keys as abstract tokens more than literal items

3 Path-Based Diagrams

- basic idea already present in PaceMaker: select path in statechart and visualize data along path
- main design guideline: build upon high configurability already present in the system
- following sections describe functionalities, implementation choices and workflow/usage of these features implemented in pixe

3.1 Path Selection

3.1.1 Functionality

- selection of nodes in statechart triggers calculation of path connecting them
- selected path is highlighted (visual feedback)
- path is subsequently used for diagram generation

3.1.2 Implementation

- select start + end node, optionally intermediate -> calculate path using dijkstra
- `pathsApi.ts`: to allow for different algorithms in the future (e.g. specific pathfinding for nestedness and concurrency)

3.1.3 Workflow

- selection of at least two nodes
 - Ctrl + click
- path is calculated along nodes **in order of selection**
- if path is found, the corresponding nodes are highlighted
- path can be de-selected by removing node selection (clicking anywhere)

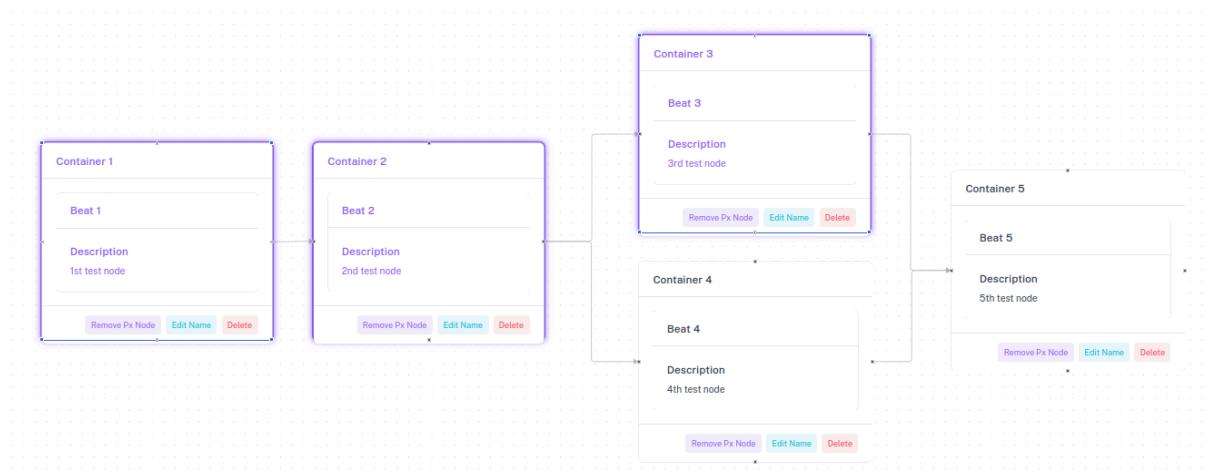


Figure 1: Path highlighted in state chart

3.2 Diagram Generation

3.2.1 Functionality

- expandable element on statechart page
 - usability: users can hide diagrams while working on statechart
- can create multiple diagrams and remove diagrams (red because diagrams do not persist)

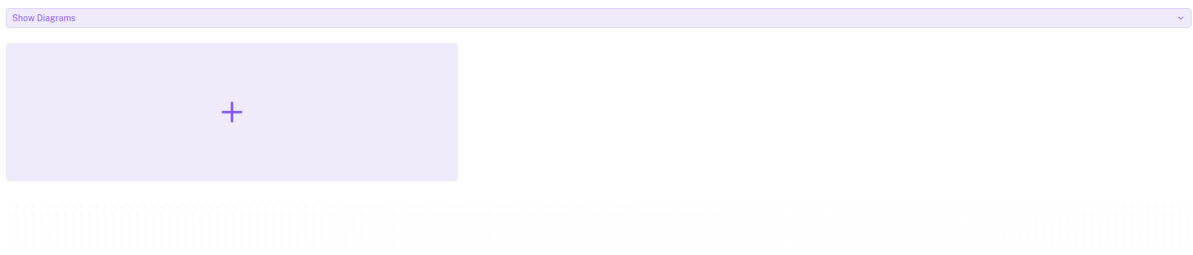


Figure 2: Upper section of the charts page with expandable element and UI for diagram creation

- selection of one or multiple component definitions for the y axis
- optionally selection of one component for the x axis (e.g. estimated playtime)
 - values are summed up along selected path
 - default: equal spacing

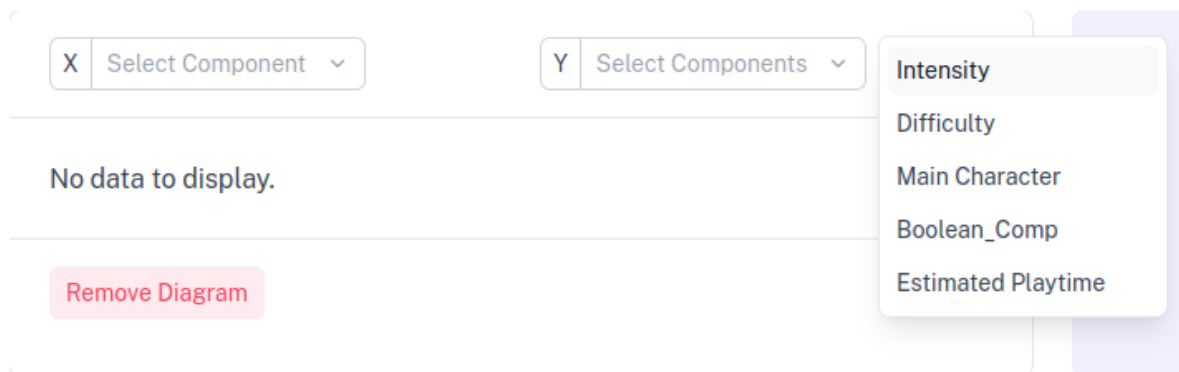


Figure 3: Selectors for components to be mapped to X/Y axes

- selected path along x axis
 - if no path selected: all nodes,  sorted



Figure 4: Diagram visualizing component along selected path

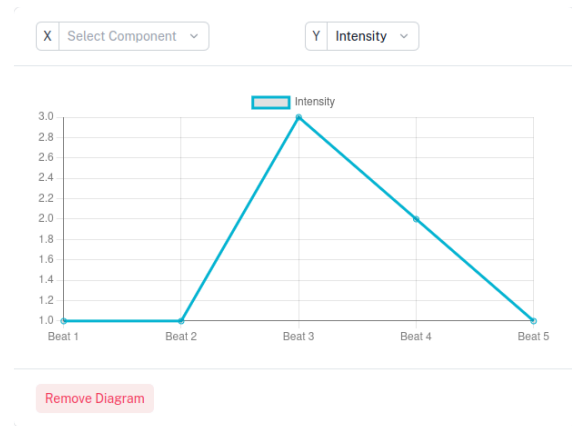


Figure 5: Diagram visualizing component across all nodes

3.2.2 Implementation

- [chartJS](#) library
- modular architecture: `PxDiagrams` as wrapper element
 - manages diagrams
 - currently: passes path (reactively), deletes diagrams
 - future work: path snapshots, different data types
- data is extracted from nodes once per diagram, switching between axis configurations is then done via different parsing configurations

3.2.3 Workflow

1. add new diagram
2. in any order:
 - select one or no component for x axis
 - select one or more component for y axis
 - select path
- changes in any of the configurations will be reflected live in the diagram

3.2.4 Limitations

- diagrams do not persist (reset when reloading page)
- only line diagrams for now
- same path selection for all diagrams

4 Locks and Keys

requirements/context:

- multiple keys possible per node
- multiple locks possible per node
- pathfinding and reachability analysis

4.1 Definition

(Ashmore and Nitsche 2007): “The puzzle is finding out what is an obstacle, what and where is a key to overcome it, and finally using the key to master the challenge.”

- in statechart: conditional transition

(Ashmore and Nitsche 2007): “Obstacles may not be passed until the player obtains some token (such as an item or skill)”“

- keys as abstract tokens, different kinds to distinguish: keys/locks should have some category to specify this

(Dormans 2017): “Some locks are barriers that might be navigated without a key, but this crossing the barrier might be uncertain or impose a certain risk.”

(Ashmore and Nitsche 2007) present lock/key puzzles as a type of quest

- future work: model/analyze other types of quests (explicit, specific tasks, mission objectives)

4.2 Taxonomy

(Dormans 2017): “When you unlock a door, that door might remain unlocked forever (permanent), for a short period of time (temporary), or until it is relocked (reversible). Sometimes, a lock collapses after use, allowing the player only to pass once.”

-> locks: **permanent, temporary, reversible, collapsible**

(Dormans 2017): “Single-purpose keys can only be used to open a lock, and for nothing else, while multipurpose keys can also be used in different ways.”

-> keys: **single-purpose or multi-purpose**

- attribute type on keys: ability, item

(Dormans 2017): “Particular keys are the only thing that unlocks a particular lock, whereas several nonparticular keys might unlock a single lock.”

-> keys: **particular or non-particular**

- probably easier to understand when this is a property of a lock, i.e. key requirement

(Dormans 2017): “Keys that are destroyed somehow in the process of unlocking a door are consumable, while keys that are not are persistent.”

-> keys: **consumable or persistent**

(Dormans 2017): “Levers and switches are the best example of keys that are fixed in place (and typically single purpose and particular as well).”

-> keys: **fixed or not**

(Dormans 2017): “Certain locks allow you to cross only in one direction (valves), while others can only be opened from one direction but traversed in two directions after they are opened (asymmetrical). [...] Valves do not always require a key.”

-> locks: **valve, asymmetrical**

- modeled via bidirectional edges with locks vs two uni-directional edges, one with lock and one without

4.2.1 Not Modeled Currently

(Dormans 2017): “A safe lock is guaranteed to have a solution, while an unsafe lock is not.”

4.2.2 Data Model Overview

 **add diagram of data model**

4.3 Creating and Visualizing Locks and Keys

(Brown 2026): high-level visual representation. only focuses on locks and keys and where they are in relation to each other

to integrate into player experience chart: assign keys to nodes and locks to edges (to maintain clear structure)

4.3.1 Modeling Keys

- similar to PxComponents: definitions separate from instantiations in nodes
- multiple instances of same definitions possible per node

 **add screenshots of key definitions and assignments in UI + explain options**

4.3.2 Modeling Locks

- same split between definitions and instantiations
- instantiations on edges: additional UI/modal

 **add screenshots of lock definitions and assignments in UI + explain options**

4.4 Pathfinding with Locks and Keys

4.4.1 Basic Dijkstra with Locks and Keys

- goal: all locks assigned to edges in the path have to be unlocked by key(s) collected up to the edge in question
- inventory: track keys collected along path to determine whether encountered edges can be unlocked
 - inventory similar to (Aversa, Sardina, and Vassos 2015)
 - modeled via `PxKeySet` -> formally: mapping of key defs to counts
- basic form: inventory associated with a node reflects keys collected along shortest path to this node and keys made available in this node -> later extended (consumable keys)

4.4.1.1 Unlocking Function

- `canUnlock()`: given (multi-)sets of locks and keys, determine whether locks can be unlocked
 - added complexity due to an edge potentially having multiple locks, and each locks potentially being unlocked by multiple keys
 - exponential blowup, but no optimization so far as average case is expected to be less complex  citation

given set of keys in inventory K and locks on edge L :

```
canUnlock(K, L):
  if L = ∅
    return TRUE
  unlockingKeysets = required(l1) × ... × required(ln)
  if ∃ U ∈ unlockingKeysets where ∀ u ∈ U. u ∈ K:
    return TRUE
  else
    return FALSE
```

4.4.1.2 Example

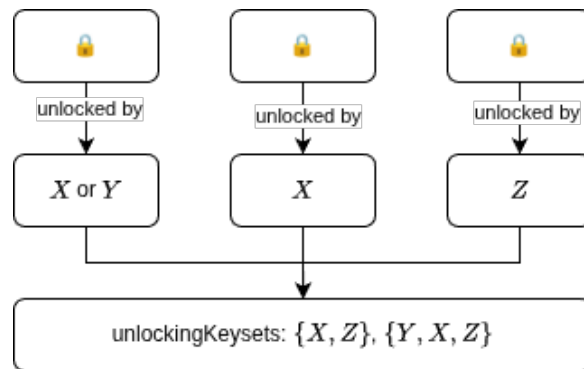


Figure 6: Calculation of unlocking keysets

- $I = \{X, Y\} \rightarrow \text{false}$
- $I = X, Y, Z \rightarrow \text{true}$

4.4.1.3 Full Algorithm

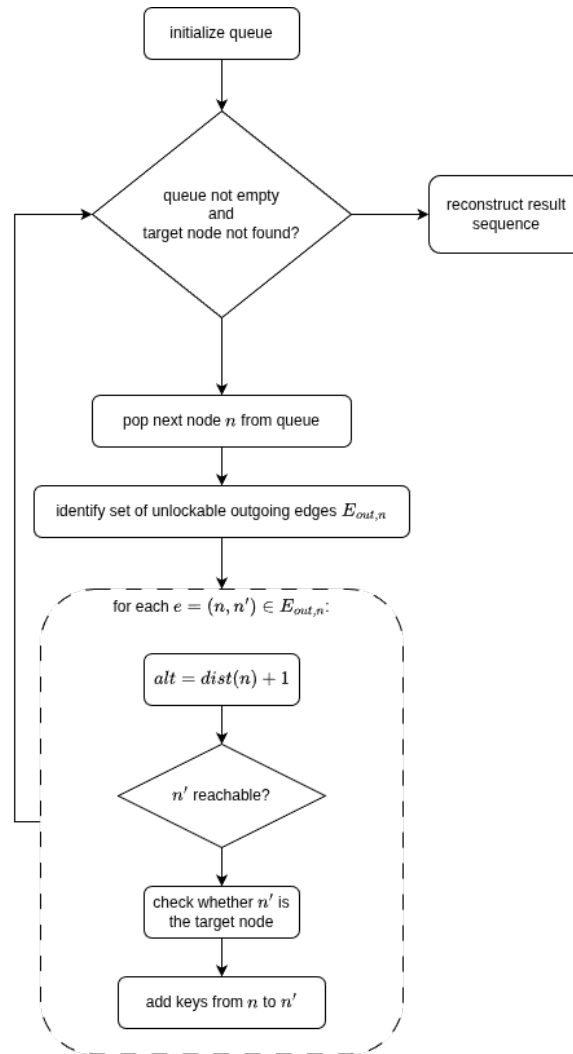


Figure 7: Calculation of unlocking keysets for key consumption

4.4.1.4 Example Iteration

-  add small example of iteration

4.4.2 Fixed Keys

- goal: fixed keys can only unlock locks on outgoing edges of the node they are assigned to

implementation:

- do not add keys to inventory of neighbors
-  add example

4.4.3 Soft Gates

- goal: soft gates **can** be passed without the corresponding key, but this should be indicated in the path highlighting

implementation:

- during pathfinding, ignore soft-gate locks
- after path has been found, re-trace it to determine whether keys are available or not
-  add example from pixe

4.4.4 Consumable Keys and Soft-Lock Detection

- goal: inventory should reflect usage of single-use keys
- variants of inventory -> idea of ‘parallel universes’
 - nodes exist in different realities if they have different inventories
 - assign each node a set of possible keysets

implementation:

- maintain set of `PxKeySets` per node
- in `canUnlock`, consider multiplicity of consumable keys
- for updating inventory of neighbor nodes: remove consumed keys from inventory
 - if there are multiple possible constellations, add multiple keysets

4.4.4.1 Pathfinding with Consumable Keys

Extending Unlocking Check

given multiset of keys in inventory K and locks on edge L :

```
canUnlock(K, L):
  if L = ∅
    return TRUE
  unlockingKeysets = required(l1) × ... × required(ln)
  if ∃ U ∈ unlockingKeysets where U ⊆ K:
    return TRUE
  else
    return FALSE
```

- note: subset of multiset

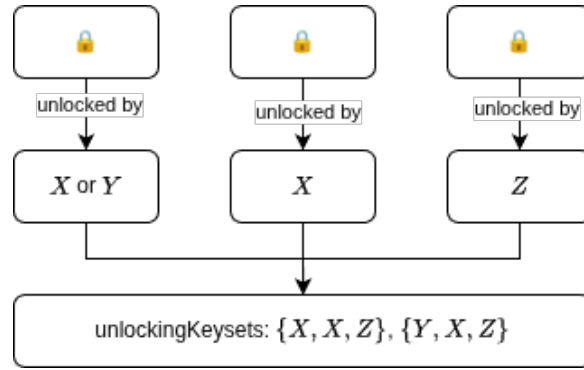


Figure 8: Calculation of unlocking keysets

- now: $I = \{X, Z\} \rightarrow \text{false}$

Removing Consumable Keys from Inventory

given inventory I containing multisets of keys K and locks on edge L :

```

canUnlock(I, L):
  if L = ∅
    return TRUE
  consumableRequirements = locks with at least one consumable key unlocking
  unlockingKeysets = required(l1) × ... × required(ln)
  updatedInventory = {}
  for K_i ∈ I:
    if ∃ U ∈ unlockingKeysets with K_i ⊆ U:
      updatedInventory.push(K_i - {key ∈ U. consumable(key)})
  return updatedInventory
  
```

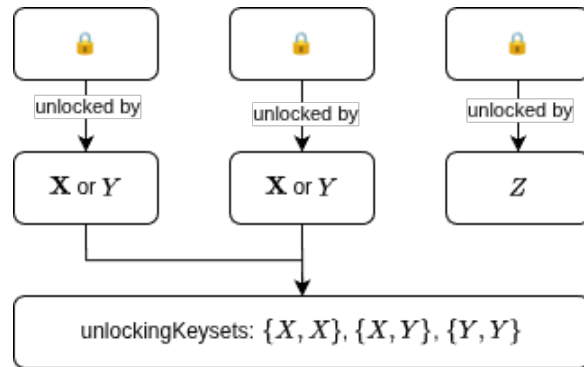


Figure 9: Calculation of unlocking keysets for key consumption

- $I = \{X, X, Z\} \rightarrow \{Z\}$

-  example in pixe for consumable keys

4.4.4.2 Soft-Locks

definition: “it is possible to reach the goal state from every reachable state” ([Mawhorter and Smith 2021](#)):

- here: not nodes, but *states* in which nodes are visited

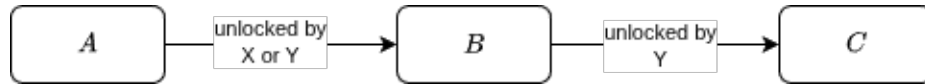


Figure 10: Calculation of unlocking keysets for key consumption

implementation:

- potential soft-lock state: node unlockable with some, but not all keysets
- when removing consumed keys: remove empty keysets entirely. then, compare inventory length before and after consumption
-  example in pixe for consumable keys

4.5 Bidirectional Edges and Backtracking

- goal: allow for longer/cyclic paths needed to collect keys
- another idea of parallel universes
-  add citation from julians paper?

implementation:

- alternative used here: nodes are re-added to queue iff they have a different inventory assigned to them than in previous encounters -> only duplicate nodes as needed
 - for each node, maintain inventories it has been visited with
 - only re-visit with a different inventory
 - add limiting constant for revisits
 - also maintain unlocked edges



Figure 11: Calculation of unlocking keysets for key consumption

- critical path: $A - B - A - C$

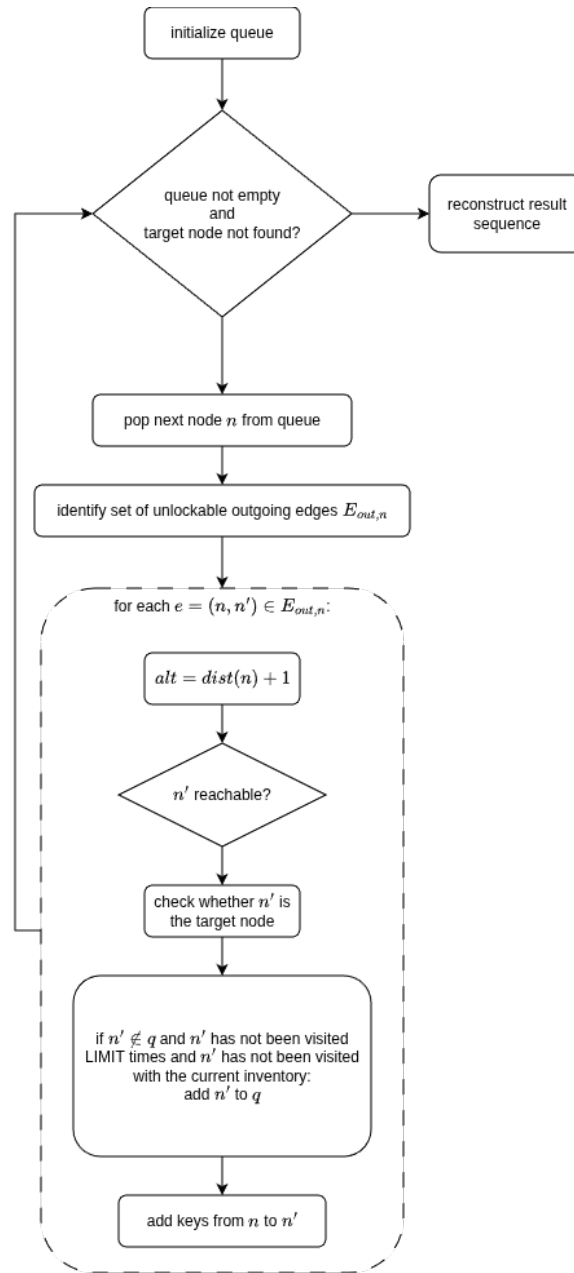


Figure 12: Calculation of unlocking keysets for key consumption

4.5.1 Example

-  example screenshot from pixe

4.5.2 Full Algorithm

-  make diagram of full algorithm?

4.6 Options/Settings

-  describe pathfinding options

4.7 Code Architecture

-  add diagram for code architecture?

4.8 Testing

-  describe

4.9 Limitations and Future Work

- fixed keys for non-adjacent locks
- in pathfinding: temporary, reversible, collapsible locks
- difficulty estimation  add citation
- recommendations for lock/key types ([Dormans 2017](#))
- LLM-based consistency check with descriptions

5 Evaluation

- user study

5.1 User Study Design

5.1.1 Overall Setup

- task-based within-subject: StatePx vs Miro
 - two task variants, varying order of tools + dungeons -> 4 groups

	Miro first	StatePx first
Variant A first	MA	PA
Variant B first	MB	PB

- use case: zelda dungeons ([Summerville et al. 2016](#)) -> ensure realistic tasks. data pool big enough to choose 2 comparable dungeons per task. same game: consistency in terms of details like item/key types, dungeon structure, etc.
- target demographic: people with some understanding of game design -> recruitment: university lecture in games engineering study course, also computer science students who are familiar with videogames
- complete tasks (all dungeons) + answer keys in appendix  **add to appendix**

5.1.2 Modeling Zelda Dungeons

5.1.2.1 General Modeling

- rooms numbered based on location in map (approx from bottom left to top right)

 **image showing room numbering**

5.1.2.2 pix:e vs Miro

pix:e: rooms: node. use in-built functionalities for keys/items and other room data (components). visualization: as described previously

Miro: rooms: sticky notes. write all information into sticky notes. heavier use of emoji encoding: to more easily differentiate between different key types/lock types

comparison: pix:e has more dedicated functionality + logic, but ui is less mature (zoomed out view, editing features). emoji encoding in Miro.

5.1.3 Solvability Tasks

- larger Zelda dungeons
- Link's Awakening LA_3, LA_4 ([Summerville et al. 2016](#))
- selection criteria: size (35 vs 41 nodes), number of keys/locks. comparable size + complexity (outgoing ranks of nodes)  **add details**
-  **images: original maps/graphs, versions in Miro/pix:e**
- slight adaptations:  **add adaptations**
 - structural:  **add structural adaptations**
 - information: remove puzzles, "vision" edges, enemies
- added issues: 6 for each dungeon, of varying difficulty
 - 2x missing key(s), 1x no outgoing edge, 1x unreachable due to edge directionality, 2x softlock
- task: find as many issues -> specified kinds of issues
- participants were allowed to use tool functionalities
 - Miro: e.g. draw in paths, use pens/sticky notes to mark found issues
 - pix:e: pathfinding functionality w/ highlighting, fixing issues after reporting

5.1.4 Pacing Analysis Tasks

- smaller Zelda dungeons
- Link's Awakening LA_1, LA_2 ([Summerville et al. 2016](#))
- selection criteria: size, number of keys/locks. comparable size + complexity  **add details**
-  **images: original maps/graphs, versions in Miro/pix:e**
- adaptations:  **add adaptations**
 - structural:  **add structural adaptations**
 - information: remove puzzles, "vision" edges
 - additional information: completion time based on YouTube walkthrough (100% completion to get data for all rooms)  **add ref to walkthroughs**
- task: answer questions about pacing. includes comparisons of path sections, overall enemy distribution in nodes:
 1. Is the number of enemies steadily increasing throughout the dungeon?
 2. Does it take longer on average to complete rooms in Section A than in Section B?
 3. Are there more enemies in Section C than in Section B?
 4. Which room most likely contains a miniboss?
 5. Between which two consecutive rooms does the number increase the most?

6. Which room after a high-intensity encounter has notably fewer enemies or lower completion time?

- participants were allowed to use tool functionalities
 - Miro: e.g. draw in paths, use pens/sticky notes to note down calculations and answers
 - pix:e: pathfinding functionality w/ highlighting, diagrams

5.1.5 Collected Data

- demographics: age, gender, background, occupation experience, previous experience with whiteboard tools
-  add complete demographics
- recorded during tasks: issues found/questions answered + timestamps, comments, clarifications/hints, observations about tool use
- additionally: UEQ-S ([Schrepp, Hinderks, and Thomaschewski 2017](#)), questionnaire comparing Miro and StatePx

5.2 User Study Results

5.2.1 Data Processing

 potentially more of a methodology section? check wiki

- data recorded in semi-structured or format -> parsing script to transform
- different sets of data:
 - demographics
 - task meta data per participant and task: total counts/points - based on protocol, may contain some human error
 - time series per participant and task: timestamp + individual answers, comments, etc - based on protocol, may also contain some human error
 - UEQ-S
 - comparison questionnaire

5.2.2 Data Analysis

5.2.2.1 Participants

- total number: 24
- demographics
- group breakdown

5.2.2.2 Results Solvability

- total number of identified issues
- correctly identified issues (f1)
- times to first identified issues

5.2.2.3 Results Pacing

- total number of answered questions
- correctly answered questions (f1)
- wrongly answered questions
- time per question

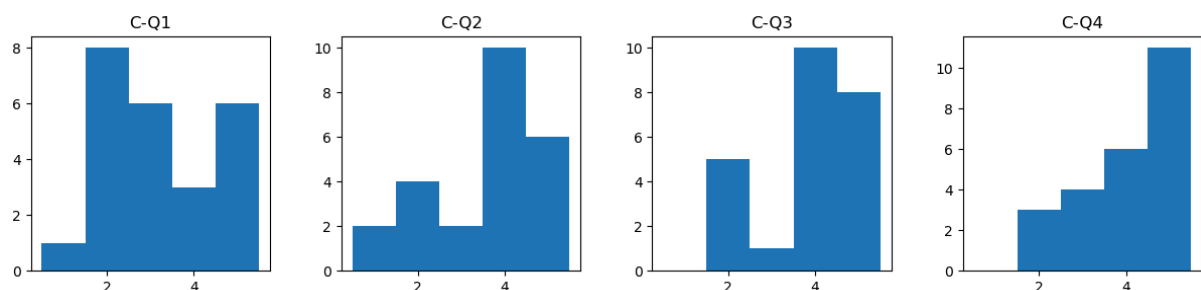
5.2.2.4 Results UEQ-S

- using provided analysis tool
- pragmatic quality (“relate to the tasks or goals the user aims to reach”): 0.823, hedonic quality (“related to pleasure or fun while using the product”): 1.385, overall: 1.104
-  contextualize
- weakest item: confusing vs clear
 - potentially relates to: UI quirks, lock visualization

5.2.2.5 Results Comparative Questions

- C-Q1: “Compared to Miro, the system made it easier to analyze dungeon designs. “
- C-Q2: “Compared to Miro, I felt more confident in my answers. “
- C-Q3: “Compared to Miro, I was able to complete the tasks more efficiently. “
- C-Q4: “If I were analyzing game levels in practice, I would prefer using the system over a general-purpose whiteboard.”

5-point Likert scale, fully disagree (1) to fully agree (5)



- overall, participants indicated preference for statepx over miro, mixed answers with a slight preference for pixie in first three questions

- in line with high hedonic quality reported in UEQ-S
- previous whiteboard experience correlated with lower preference for pix:e  back this up with calculation

5.2.2.6 Validity of A/B Test

- compare results across groups -> statistical test
- compare demographics across groups

5.3 Limitations

- data collection: human errors
- not all functionality included (e.g. soft gates)

5.4 Post-Study Implementations

5.4.1 Reachability Analysis

- many participants (number) either explicitly reported missing a reachability analysis functionality in open feedback or were observed to conduct a reachability analysis during the solvability task by selecting various end nodes for the given start node and subsequently identifying locked edges as those between a node reachable from the start and a node not reachable from the start.  add numbers
- dijkstra algorithm tracks distances of nodes from start nodes. initial value: `Infinity`, i.e., all nodes with distance < `Infinity` are reachable. -> simply filter after pathfinding has concluded
- in case of failed pathfinding: highlight unreachable nodes  confirm/wait for julians pr feedback

 add screenshots

5.4.2 Increased Contrast of Nodes Against Background

- high number of participants reported low contrast between nodes and background as a usability issue, especially when zoomed out  add numbers
- new color in css theme for background for both dark and light mode

 add screenshots

Bibliography

- Calvin Ashmore and Michael Nitsche. 2007. The Quest in a Generated World. In *Proceedings of DiGRA 2007 Conference: Situated Play*.
- Davide Aversa, Sebastian Sardina, and Stavros Vassos. 2015. Path Planning with Inventory-Driven Jump-Point-Search. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 11(1):2–8.
- Mark Brown. 2026. How My Boss Key Dungeon Graphs Work | Patreon. <https://web.archive.org/web/20260226034644/https://www.patreon.com/posts/how-my-boss-key-13801754>.
- Joris Dormans. 2017. Cyclic Generation.
- Julian Geheeb, Daniel Dyrda, and Sebastian Geheeb. 2024. PaceMaker: A Practical Tool for Pacing Video Games. In *2024 IEEE Conference on Games (CoG)*, pages 1–8.
- David Harel. 1987. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3):231–274.
- Ross Mawhorter and Adam Smith. 2021. Softlock Detection for Super Metroid with Computation Tree Logic. In *Proceedings of the 16th International Conference on the Foundations of Digital Games*, pages 1–10, New York, NY, USA. Association for Computing Machinery.
- Martin Schrepp, Andreas Hinderks, and Jörg Thomaschewski. 2017. Design and Evaluation of a Short Version of the User Experience Questionnaire (UEQ-S). *International Journal of Interactive Multimedia and Artificial Intelligence*, 4(6):103–108.
- Adam James Summerville, Sam Snodgrass, Michael Mateas, and Santiago Ontañón. 2016. The VGLC: The Video Game Level Corpus.